МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»

Кафедра інженерії програмного забезпечення

КУРСОВА РОБОТА

(ПОЯСНЮВАЛЬНА ЗАПИСКА)

з дисципліни: «Backend-розробка»

на тему:

«Онлайн-платформа для ведення щоденника»

студента III курсу групи ІПЗ-23-4

спеціальності 121 «Інженерія програмного забезпечення»

Отовчиця Іллі Сергійовича

(прізвище, ім'я та по-батькові)

Керівник ст. викладач кафедри ІПЗ

Ірина ДМИТРЕНКО

Дата захисту: "____" _____ 2025 р.

Національна шкала

_____

Кількість балів:

_____

Оцінка: ECTS

_____

Члени комісії _____ Валентин КОВТУН

_____

(підпис) (прізвище та ініціали)

_____ Олена ЧИЖМОТРЯ

(підпис) (прізвище та ініціали)

_____ Ірина ДМИТРЕНКО

(підпис) (прізвище та ініціали)

Житомир – 2025

ДЕРЖАВНИЙ УНІВЕРСИТЕТ «ЖИТОМИРСЬКА ПОЛІТЕХНІКА»

Факультет інформаційно-комп'ютерних технологій

Кафедра інженерії програмного забезпечення

Освітній рівень: бакалавр

Спеціальність 121 «Інженерія програмного забезпечення»

«ЗАТВЕРДЖУЮ»

Завідувач кафедри ІПЗ

_____ Тетяна ВАКАЛЮК

«____»_____2025 р.

ЗАВДАННЯ

НА КУРСОВУ РОБОТУ СТУДЕНТУ

Волинцю Андрію Сергійовичу

1. Тема роботи: «Онлайн-платформа для ведення щоденника»

   керівник роботи: ст. викладач кафедри ІПЗ Ірина ДМИТРЕНКО

2. Строк подання студентом: « 30 » березня 2026 р.
3. Вихідні дані до роботи: Розробити сайт для ведення щоденника з власною системою керування вмістом (CMS).
4. Зміст розрахунково-пояснювальної записки (перелік питань, які підлягають розробці)

   1. Постановка завдання
   2. Аналіз аналогічних розробок
   3. Алгоритми роботи програми
   4. Опис роботи програми
   5. Програмне дослідження

5. Перелік графічного матеріалу (з точним зазначенням обов'язкових креслень)

   1. Презентація до КР

2. Посилання на репозиторій: https://github.com/shino-bit/diary_01.git

6. Консультанти розділів роботи

Розділ	Прізвище, ініціали та посади консультанта	Підпис, дата	
		завдання видав	завдання прийняв
1-3	Ірина ДМИТРЕНКО	01.03.2025	01.03.2025

7. Дата видачі завдання “ 01 ” березня 2025 р.

КАЛЕНДАРНИЙ ПЛАН

з/п	Назва етапів курсового проєктування	Строк виконання етапів роботи	Примітки
	Постановка задачі	01.03.2025	Виконано
	Пошук, огляд та аналіз аналогічних розробок	16.03.2026	Виконано
	Формулювання технічного завдання	17.03	Викон

		.2026	ано
	Опрацювання літературних джерел	18.03 .2026	Виконано
	Проєктування структури	18.03 .2026	Виконано
	Написання програмного коду	25.03 .2026	Виконано
	Відлагодження	28.03 .2026	Виконано
	Написання пояснювальної записки	30.03 .2026	Виконано
	Захист	13.06 .2025	

Студент _____ Ілля ОТОВЧИЦЬ

(підпис) (прізвище та ініціали)

Керівник роботи _____ Ірина ДМИТРЕНКО

(підпис) (прізвище та ініціали)

РЕФЕРАТ

Курсова робота присвячена розробці веб-додатка Diary Система ведення особистого щоденника на базі мови програмування PHP, СУБД MySQL та власної реалізації архітектурного патерну MVC. Метою роботи є створення захищеної та зручної інформаційної системи, що дозволяє користувачам здійснювати повний цикл керування особистими нотатками: від створення та редагування до адміністрування облікових записів. У межах роботи виконано аналіз предметної області, розроблено технічне завдання, спроєктовано структуру бази даних та реалізовано ядро фреймворку з механізмом динамічної маршрутизації та обробки запитів.

Пояснювальна записка містить вступ, три розділи, висновки, список використаних джерел і додатки. У першому розділі проаналізовано існуючі рішення для ведення цифрових записів, сформульовано вимоги до системи та обґрунтовано вибір технологічного стеку. У другому розділі виконано проєктування архітектури системи, розроблено схему бази даних Entity-Relationship та реалізовано базові класи фреймворку. У третьому розділі завершено розробку користувацьких інтерфейсів, впроваджено систему адаптивної стилізації та проведено тестування функціональних модулів. Реалізовано модулі: реєстрація та авторизація за Email, стрічка нотаток у вигляді карток, редактор записів та панель адміністратора для керування користувачами.

Ключові слова: ЩОДЕННИК, PHP, MYSQL, MVC, ФРЕЙМВОРК, КОРИСТУВАЧ, CONTROLLER, ВЕБ-ДОДАТОК, НОТАТКИ, АВТОРИЗАЦІЯ.

ЗМІСТ

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ 1

ПЕРЕЛІК УМОВНИХ СКОРОЧЕНЬ, СЛОВНИК 9

ВСТУП 10

РОЗДІЛ 1. АНАЛІЗ ПРОБЛЕМАТИКИ, МЕТОДІВ ТА ЗАСОБІВ ВИРІШЕННЯ ЗАДАЧІ 12

1.1 Аналіз задачі, засобів та методів її вирішення 12

1.2 Аналіз існуючих веб-сайтів за тематикою курсової роботи 13

1.3 Технічне завдання на курсову роботу. 15

1.3.1 Загальне положення 15

1.3.2 Призначення розробки та область застосування 16

1.3.3 Найменування розробника та замовника. 16

1.4 Підстава для розробки 16

1.4.1 Документ на підставі якого ведеться розробка 16

1.5 Вимоги до програми 16

1.5.1 Вимоги до функціональних характеристик. 17

1.5.1.1 Загальні вимоги 17

1.5.2. Склад виконуваних функцій 17

1.5.3. Організація вхідних і вихідних даних 18

1.5.4. Часові характеристики і розмір пам'яті, необхідної для роботи програми. 18

1.5.5 Вимоги до методів рішення і мов програмування 18

1.5.5.1. Вимоги до системи програмних засобів. 18

Висновки до першого розділу 19

РОЗДІЛ 2. ПРОЕКТУВАННЯ ТА РОЗРОБКА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ 20

2.1. Проектування загального алгоритму роботи програми 20

2.2. Розробка функціональних алгоритмів роботи програми 22

2.3. Розробка програмного забезпечення 24

2.4 Реалізація архітектурного ядра вебдодатку 25

Висновки до другого розділу 29
РОЗДІЛ 3. ОПИС РОБОТИ З ПРОГРАМНИМ ДОДАТКОМ ТА ЙОГО ТЕСТУВАННЯ..... 31
3.1. Опис роботи з програмним додатком 31
3.2. Тестування роботи програмного забезпечення..... 35
Висновок третього розділу..... 37
ВИСНОВКИ 38
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ 39
ДОДАТКИ..... 40

ПЕРЕЛІК УМОВНИХ СКОРОЧЕНЬ, СЛОВНИК

БД - база даних.

ПЗ - програмне забезпечення.

Рис. - рисунок.

СУБД - система управління базами даних.

CSS (Cascading Style Sheets) - каскадні таблиці стилів.

HTML (HyperText Markup Language) - мова розмітки веб-сторінок.

MVC (Model-View-Controller) - архітектурний патерн розділення логіки додатка.

ВСТУП

Актуальність теми. В умовах стрімкої цифровізації суспільства та постійного збільшення обсягів інформації, потреба у зручних інструментах для фіксації особистих думок, ідей та завдань стає дедалі гострішою. Традиційні паперові щоденники поступаються місцем електронним аналогам через обмеженість доступу, відсутність пошуку та ризики втрати даних. Створення персонального вебдодатка-щоденника дозволяє користувачам мати безпечний і швидкий доступ до своїх записів з будь-якого пристрою, підключеного до мережі Інтернет.

У сучасній веброзробці критично важливою є не лише наявність інтуїтивно зрозумілого інтерфейсу, а й побудова надійної та захищеної серверної архітектури. Розробка власної системи ведення нотаток на базі власного MVC-фреймворку надає гнучкість, незалежність від сторонніх платформ (CMS) та гарантує повний контроль над конфіденційними даними користувачів.

Мета і завдання роботи. Метою курсової роботи є розробка вебдодатка Diary особистий щоденник з функціоналом безпечної реєстрації, керування власними нотатками та адміністрування системи.

Для досягнення цієї мети необхідно виконати такі завдання:

проаналізувати предметну область та визначити вимоги до структури та функціоналу вебдодатка;

спроєктувати базу даних для зберігання облікових записів та контенту;

реалізувати серверну частину мовою PHP з використанням архітектурного підходу MVC;

розробити інтерфейс користувача з використанням модульної архітектури CSS та HTML5;

створити панель адміністратора для керування користувачами системи.

Об'єкт дослідження – методи, моделі та інструменти розробки сучасних вебзастосунків на основі архітектурного патерну MVC.

Предмет дослідження - процеси проєктування бази даних, маршрутизації та програмної реалізації вебдодатка Diary із застосуванням мови PHP та СУБД MySQL.

РОЗДІЛ 1. АНАЛІЗ ПРОБЛЕМАТИКИ, МЕТОДІВ ТА ЗАСОБІВ ВИРІШЕННЯ ЗАДАЧІ

1.1 Аналіз задачі, засобів та методів її вирішення

Для реалізації курсового проєкту було поставлено задачу створення вебдодатка Diary система ведення особистого щоденника із базовим функціоналом для користувачів (реєстрація, авторизація, створення та редагування нотаток) та адміністративною панеллю для керування обліковими записами. Ключовою особливістю проєкту стала повна відмова від готових рішень на кшталт комерційних CMS чи важких сторонніх фреймворків. Натомість було реалізовано власну легковагову архітектуру з використанням патерну MVC.

Основою серверної логіки обрано мову програмування PHP, яка забезпечила об'єктно-орієнтовану реалізацію модулів та чітке розмежування логіки моделей, контролерів та представлень views. У межах проєкту було розроблено власну систему маршрутизації на основі регулярних виразів (Regex), механізм обробки сесій для авторизації користувачів та систему захисту даних. Середовище розробки - локальний стек WAMP Apache, MySQL, PHP, який надав стабільну базу для розгортання реляційної бази даних MySQL. Взаємодія з базою даних здійснюється через об'єктно-орієнтований інтерфейс PDO, що забезпечує захист від SQL-ін’єкцій за допомогою підготовлених запитів Prepared Statements.

Клієнтська частина реалізована за допомогою стандартів HTML5 та JavaScript. На відміну від типових рішень на базі сторонніх бібліотек, для побудови інтерфейсу було застосовано авторську модульну систему CSS3. Особливістю фронтенд-частини є механізм «розумного завантаження» стилів через головний шаблон header.php, що дозволяє динамічно підключати лише ті CSS-файли, які необхідні для поточної сторінки. Це забезпечило високу швидкість рендерингу та відсутність конфліктів у стилізації різних модулів системи.

У процесі проєктування особливу увагу приділено безпеці персональних даних: впроваджено сучасні алгоритми хешування паролів BCRYPT та механізми валідації введених користувачем даних Email, логін. Самостійна реалізація архітектури дозволила забезпечити повний контроль над життєвим циклом запиту та створити масштабовану систему, готову до подальшого розширення функціоналу.

1.2 Аналіз існуючих веб-сайтів за тематикою курсової роботи.

На сьогодні ринок програмного забезпечення пропонує широкий вибір інструментів для цифрового ведення записів. Для порівняльного аналізу було обрано три популярні сервіси, що мають різні підходи до організації контенту: Evernote, Notion та Google Keep.

Evernote - один із найстаріших сервісів для зберігання заміток. Його перевагами є потужна система тегів, можливість прикріплення мультимедійних файлів та синхронізація між пристроями. Проте для багатьох користувачів інтерфейс Evernote став занадто перевантаженим функціями, якими вони не користуються. Крім того, більшість критично важливих функцій офлайн-доступ, великий обсяг сховища доступні лише у платній версії, що робить його менш привабливим для персонального використання студентами.

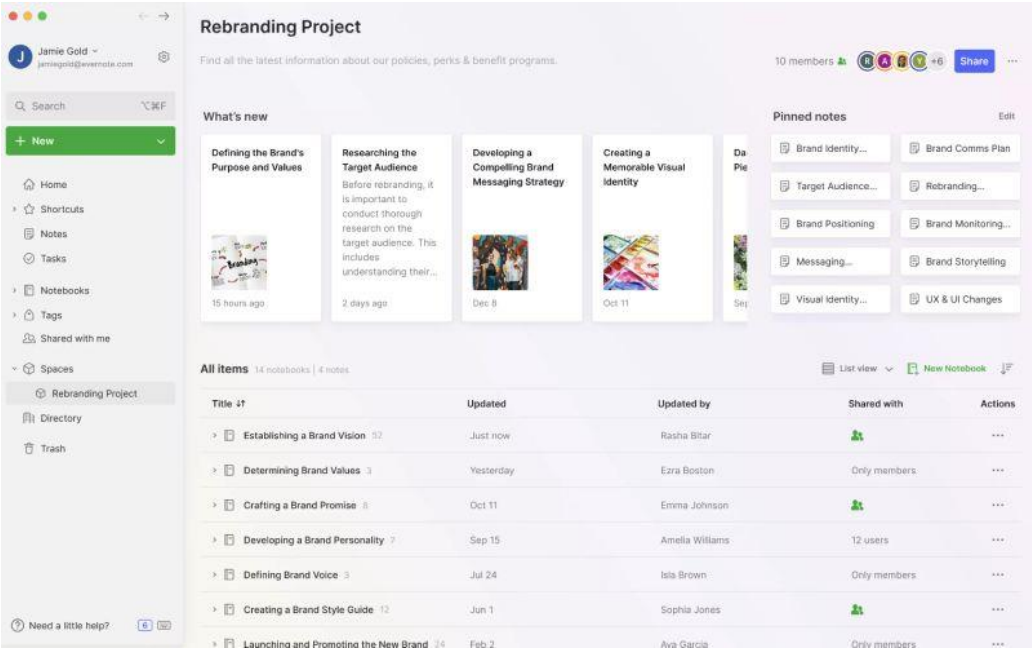

Рис 1.1

Notion - багатофункціональна платформа, що поєднує в собі щоденник, базу даних та менеджер завдань. Головна перевага - надзвичайна гнучкість у побудові сторінок. Однак Notion має високий поріг входження: користувачеві потрібно витратити значний час на вивчення інструментарію, щоб просто почати вести щоденник. Також архітектура Notion є досить важкою для веббраузерів, що призводить до повільного завантаження сторінок на пристроях із середньою продуктивністю.

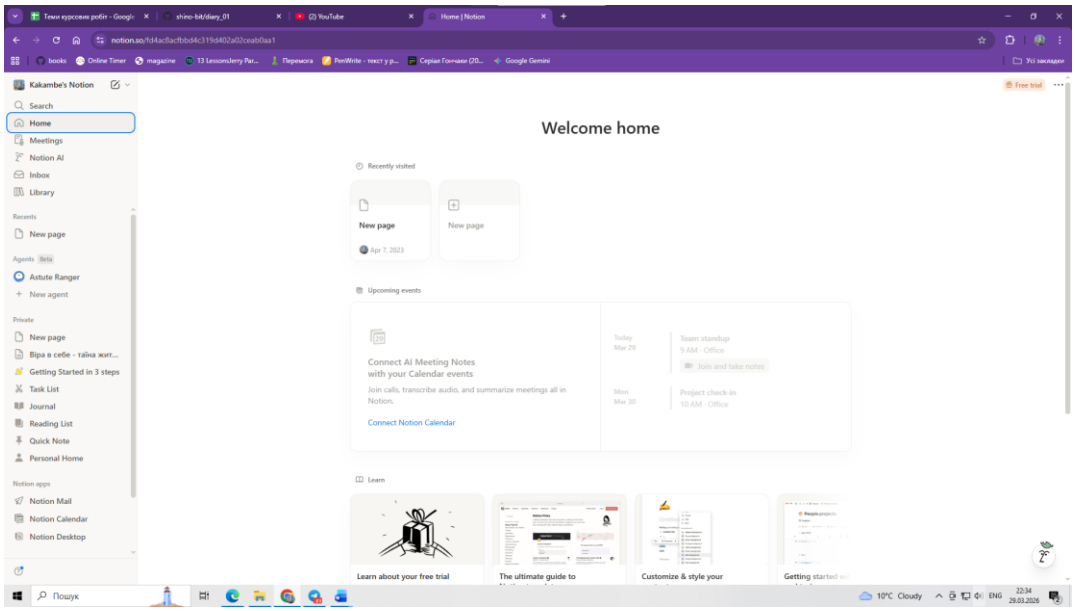

Рис 1.2

Google Keep - максимально спрощений сервіс для швидких нотаток у вигляді карток. Він ідеально підходить для коротких записів, але абсолютно не пристосований для ведення структурованого щоденника з розлогим текстовим вмістом. Відсутність форматування тексту та неможливість глибокої персоналізації інтерфейсу обмежують його використання як повноцінної системи ведення особистих хронік.

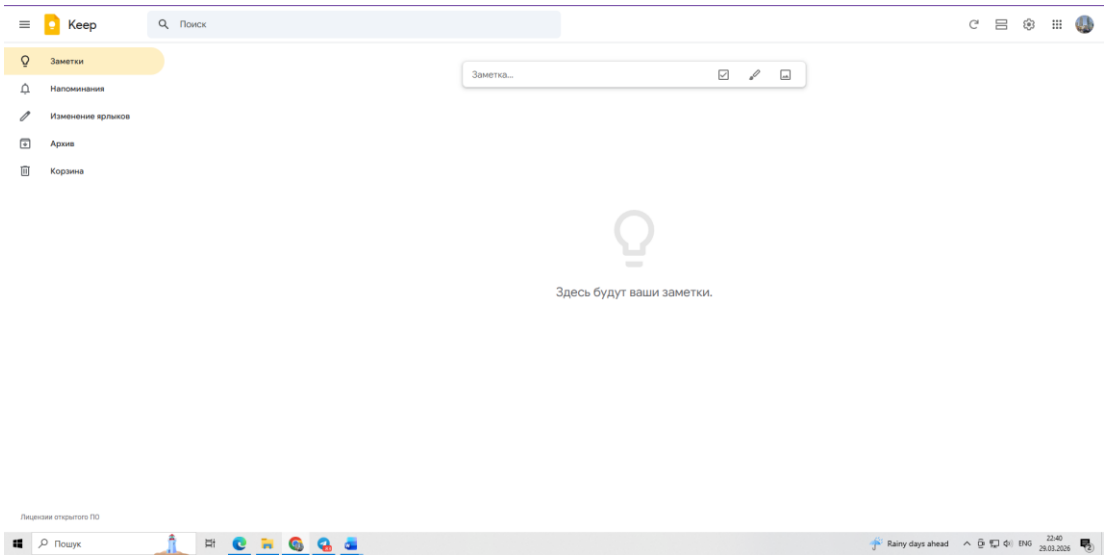

Рис 1.3

Порівняльний аналіз аналогів наведено у таблиці 1.1.

Таблиця 1.1 Порівняльна характеристика існуючих рішень

Критерії	Evernote	Notion	Google Keep
Складність інтерфейсу	Висока	Дуже висока	Низька
Гнучкість налаштувань	Середня	Максимальна	Мінімальна
Швидкість роботи	Середня	Низька	Висока
Вартість повного функціоналу	Платна	Платна для команд	Безкоштовна

1.3 Технічне завдання на курсову роботу.

1.3.1 Загальне положення

Повне найменування програмної системи: Diary-вебдодаток для ведення особистого щоденника надалі - вебдодаток. Коротка назва: Diary.

1.3.2 Призначення розробки та область застосування

Вебдодаток призначений для створення персонального цифрового простору, що дозволяє користувачам безпечно зберігати, редагувати та систематизувати власні нотатки. Система забезпечує повний цикл роботи з контентом: від реєстрації та авторизації користувача до управління архівом записів.

Адміністративна частина надає можливості моніторингу системи, перегляду списку зареєстрованих користувачів та управління їхніми обліковими записами видалення, зміна ролей.

Програмний продукт орієнтований на індивідуальне використання особами, які потребують надійного інструменту для ведення записів без залучення сторонніх хмарних сервісів. Розробка забезпечує стабільну роботу на локальному WAMP або віддаленому сервері з можливістю подальшого масштабування додавання медіафайлів, категорій тощо.

1.3.3 Найменування розробника та замовника.

Розробник продукту - студент групи ІПЗ-23-4 Отовчиць Ілля надалі - розробник.

Замовник програмного продукту - кафедра інженерії програмного забезпечення Державного університету «Житомирська політехніка» в межах виконання курсової роботи з дисципліни Backend-розробка.

Керівник: ст. викладач кафедри ІПЗ Ірина Дмитренко.

1.4 Підстава для розробки

1.4.1 Документ на підставі якого ведеться розробка

Робота ведеться на підставі навчального плану за напрямом 121 «Інженерія програмного забезпечення».

1.5 Вимоги до програми

1.5.1 Вимоги до функціональних характеристик.

1.5.1.1 Загальні вимоги

Вебдодаток має забезпечувати:

- можливість дистанційної роботи з робочих станцій через браузер у локальних та глобальних мережах;
- постійний цілодобовий доступ користувачів до вебдодатка та їх персональних даних;
- механізм розмежування прав доступу користувач/адміністратор;
- організацію управління контентом та користувачами через адміністративний інтерфейс;
- стабільну взаємодію з базою даних для збереження та отримання інформації.

1.5.2. Склад виконуваних функцій

Розроблений вебдодаток Diary підтримує виконання таких основних функцій:

- Реєстрація нових облікових записів та автентифікація існуючих користувачів.
- Відображення списку особистих нотаток у вигляді адаптивної сітки карток.
- Створення нових записів щоденника з вказанням заголовка та текстового вмісту.
- Перегляд детальної інформації окремої нотатки.
- Редагування раніше створених записів із оновленням даних у базі.
- Видалення нотаток із інтерфейсу користувача.
- Захищений вихід із системи руйнування сесії.
- Адміністрування системи: перегляд списку всіх зареєстрованих користувачів.

- Можливість видалення користувачів адміністратором у разі порушення правил сервісу.

1.5.3. Організація вхідних і вихідних даних

Вхідними даними є текстова інформація нотаток заголовок, зміст, реєстраційні дані логін, email, пароль та технічні параметри запитів ідентифікатори записів. Введення даних здійснюється через діалогові екранні форми, побудовані на основі візуальних компонентів HTML5. Форми реалізовані відповідно до логіки сайту: форма входу, форма реєстрації, інтерфейс редагування нотатки. Передбачена обов’язкова валідація даних на стороні сервера: перевірка на порожні поля, коректність формату електронної пошти та безпечне хешування паролів перед збереженням у БД.

1.5.4. Часові характеристики і розмір пам'яті, необхідної для роботи програми.

Час реакції програми на дії користувача кліки, введення даних не повинен перевищувати 0,25 с. Час завантаження сторінок меню та переходу між розділами - не більше 1 с. Доступність бази даних має становити не менше 99% часу. Операції з’єднання з БД через PDO не повинні перевищувати 0,5 с. Обсяг оперативної пам'яті, необхідний для стабільної роботи серверної частини - від 512 Мб. Дисковий простір для збереження програмного коду та файлів бази даних - не більше 200 Мб на початковому етапі.

1.5.5 Вимоги до методів рішення і мов програмування

Вибір методів рішення здійснюється розробником на основі архітектурного патерну MVC.

1.5.5.1. Вимоги до системи програмних засобів.

Серверна частина: WampServer Apache, PHP 8.x, MySQL.

Клієнтська частина: HTML5, CSS3 модульне підключення, JavaScript.

Вимоги до програмного забезпечення робочої станції розробника: Visual Studio Code.

Висновки до першого розділу

У першому розділі було здійснено всебічний аналіз задачі та сучасних підходів до розробки вебзастосунків для ведення особистих записів. Розглянуто світові аналоги, такі як Evernote, Notion та Google Keep, що дозволило виявити їхні переваги та недоліки, зокрема надмірну функціональну складність або закритість архітектури.

Проведений порівняльний аналіз допоміг виокремити основні компоненти, які мають бути реалізовані в межах курсової роботи для створення легкого та ефективного інструменту керування нотатками. Обґрунтовано доцільність розробки власного MVC-фреймворку на мові PHP, що забезпечує повний контроль над безпекою даних та логікою маршрутизації без використання сторонніх CMS чи важких зовнішніх бібліотек.

Окреслено ключові вимоги до клієнтської частини адаптивність, модульність CSS та серверної частини безпека сесій, валідація даних, використання PDO. Сформоване технічне завдання дозволяє перейти до наступного етапу - проєктування структури бази даних, формування архітектури файлової системи та безпосередньої програмної реалізації проєкту.

РОЗДІЛ 2. ПРОЕКТУВАННЯ ТА РОЗРОБКА ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ

2.1. Проектування загального алгоритму роботи програми

На етапі проєктування було визначено основні ролі користувачів, їхні функціональні можливості та базову логіку взаємодії з вебдодатком Diary. Це забезпечує структуровану обробку даних та розмежування прав доступу до персональної інформації.

Ролі та можливості.

Неавторизований відвідувач може:

- Переглядати головну сторінку.
- Перейти до форми реєстрації нового облікового запису.
- Перейти до форми авторизації для входу в систему.
- Зареєстрований користувач може:
- Переглядати власну стрічку нотаток.
- Створювати нові записи щоденника (заголовок та текст).
- Редагувати вміст раніше створених нотаток.
- Видаляти власні записи з підтвердженням дії.
- Виходити з системи завершення сеансу.

Адміністратор системи може:

- Користуватися всіма функціями звичайного користувача.
- Переглядати повний список зареєстрованих у системі користувачів.
- Бачити технічну інформацію ID, Email, роль, дата реєстрації.
- Видаляти облікові записи користувачів у разі необхідності.

Загальний алгоритм роботи програми:

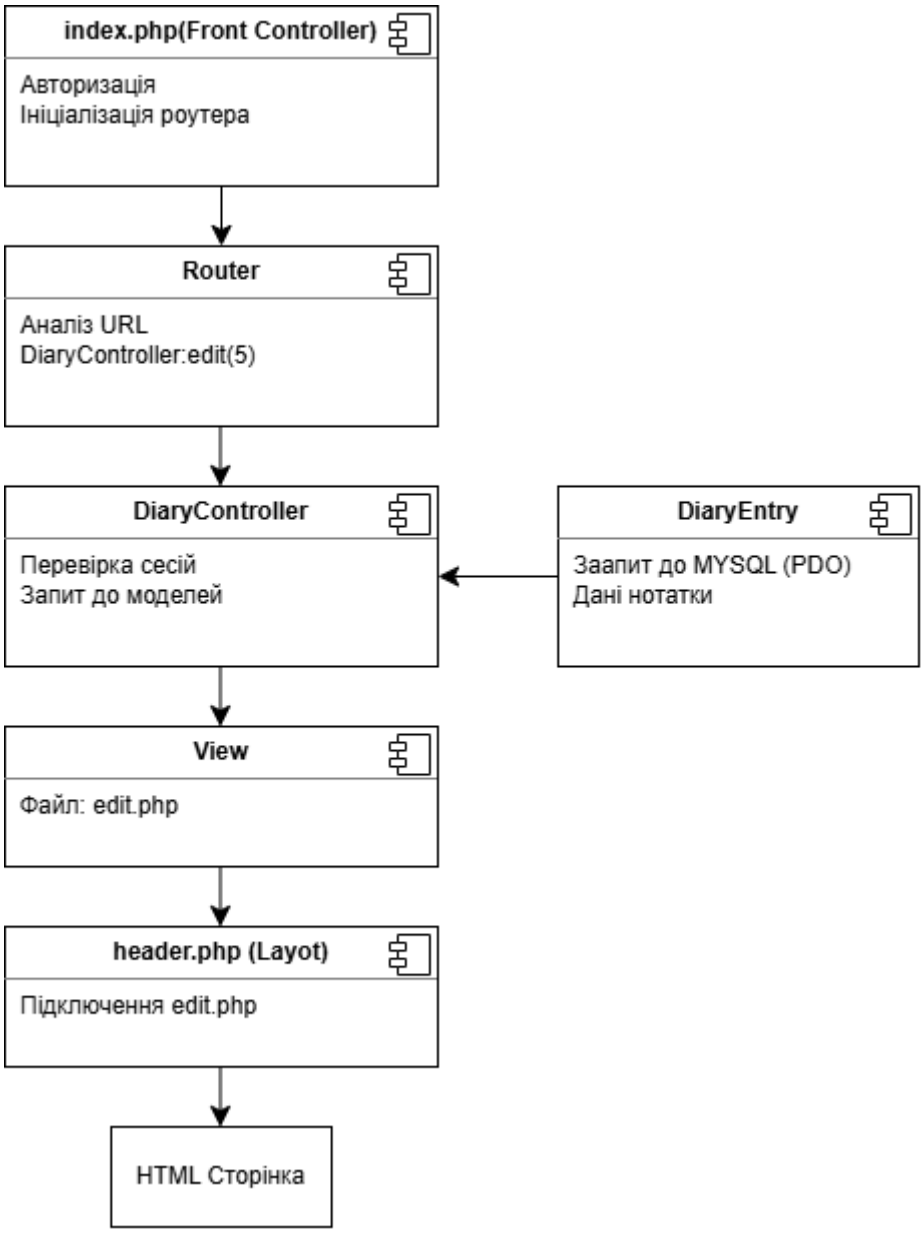

Рис. 2.1. Схема роботи MVC-фреймворку додатка Diary

Текстовий опис алгоритму:

- Надходження запиту. Користувач ініціює HTTP-запит GET або POST через браузер або шляхом відправки HTML-форми.
- Маршрутизація. Точка входу index.php передає запит до об'єкта Router. Роутер за допомогою регулярних виразів розбирає URL, визначає відповідний контролер наприклад, DiaryController та метод edit, store, index, передаючи йому необхідні параметри наприклад, ID нотатки.

- Перевірка існування ресурсу. Якщо маршрут не знайдено в таблиці маршрутизації, система генерує помилку 404.
- Перевірка доступу. Перед виконанням дії контролер перевіряє стан сесії $_SESSION. Якщо користувач намагається отримати доступ до нотаток без авторизації, система перенаправляє його на сторінку входу код 401. Якщо звичайний користувач намагається зайти в адмін-панель - доступ блокується код 403.
- Виконання бізнес-логіки. Контролер звертається до відповідної моделі User або DiaryEntry. Модель через об'єкт PDO виконує SQL-запити до бази даних MySQL для отримання, запису або оновлення інформації.
- Формування відповіді. Отримані від моделі дані наприклад, масив нотаток передаються у відповідне представлення View. Метод render() підключає необхідні файли шаблонів.
- Глобальна обробка та вивід. Через header.php динамічно підключаються CSS-стилі, після чого сформована HTML-сторінка надсилається клієнту.

2.2. Розробка функціональних алгоритмів роботи програми

Розробка логіки вебдодатка Diary здійснювалася на основі власної реалізації архітектури MVC, яка забезпечує чітке розділення відповідальності: моделі відповідають за доступ до даних, контролери - за обробку запитів та бізнес-логіку, а представлення - за формування кінцевого інтерфейсу користувача. Такий підхід дозволив створити масштабовану систему з чистою структурою коду, де кожен компонент виконує свою роль.

Кожна функціональна можливість системи реалізована у вигляді окремих контролерів, які координують взаємодію між базою даних MySQL та візуальними шаблонами.

Контролери:

- HomeController - обробка запитів до головної сторінки, відображення загальної інформації про сервіс.
- DiaryController - основний контролер для роботи з нотатками: публічний перегляд списку записів у вигляді карток, перегляд окремої нотатки, а також методи створення, редагування та видалення.
- AuthController - керування процесами реєстрації нових користувачів, автентифікації вхід у систему та завершення сеансу вихід.
- AdminController - адміністративна частина для перегляду повного списку користувачів системи та керування їхніми обліковими записами.

Моделі:

- DiaryEntry - логіка роботи з таблицею нотаток у БД: виконання CRUD-операцій Create, Read, Update, Delete через інтерфейс PDO.
- User - робота з даними користувачів: перевірка унікальності Email при реєстрації, хешування паролів за допомогою BCRYPT, перевірка прав доступу ролей.
- Database - реалізація патерну Singleton для створення єдиного захищеного підключення до бази даних MySQL.

Views:

- views/home/ - шаблони головної сторінки додатка.
- views/diary/ - список нотаток, детальний перегляд запису, форми створення та редагування контенту.
- views/auth/ - екранні форми для реєстрації та входу користувачів.
- views/admin/ - інтерфейс панелі адміністратора зі списком користувачів.
- views/layouts/ - спільні компоненти системи: header.php з логікою динамічного підключення CSS та footer.php.

Усі запити обробляються через єдину точку входу index.php, де об'єкт Router викликає потрібний метод контролера залежно від сегментів URL та HTTP-методу. Контролер виконує необхідну перевірку прав доступу, звертається до відповідної моделі для маніпуляції даними, після чого повертає

результат через представлення. Така структура забезпечує зрозумілу та логічну взаємодію між частинами системи, полегшує налагодження коду та дозволяє швидко впроваджувати новий функціонал.

2.3. Розробка програмного забезпечення

На цьому етапі було виконано структурне проєктування реляційної бази даних для вебдодатка Diary. Метою проєктування було створення мінімалістичної, але продуктивної та безпечної моделі даних, що забезпечує швидкий доступ до персональних нотаток та надійне збереження облікових записів користувачів.

Основні кроки проєктування:

1. Визначення основних сутностей, які відповідають функціональним модулям системи:

- users - облікові записи користувачів логін, email, хешований пароль та роль користувача.
- diary_entries - основна таблиця для зберігання контенту щоденника заголовки, тексти нотаток, дата створення.

2. Проєктування зв’язків між таблицями:

- В основі бази даних лежить зв'язок один-до-багатьох між таблицями users та diary_entries.
- Один користувач user може мати необмежену кількість нотаток entries.
- Кожна нотатка жорстко прив'язана до свого власника через зовнішній ключ user_id, що забезпечує ізоляцію даних: користувач отримує доступ лише до тих записів, які належать його ідентифікатору.

3. Опис полів основних таблиць:

- Таблиця users: id (INT, PK), username (VARCHAR), email (VARCHAR, UNIQUE), password (VARCHAR), role (ENUM: 'user', 'admin'), created_at (TIMESTAMP).
- Таблиця diary_entries: id (INT, PK), user_id (INT, FK), title (VARCHAR), content (TEXT), updated_at (DATETIME).

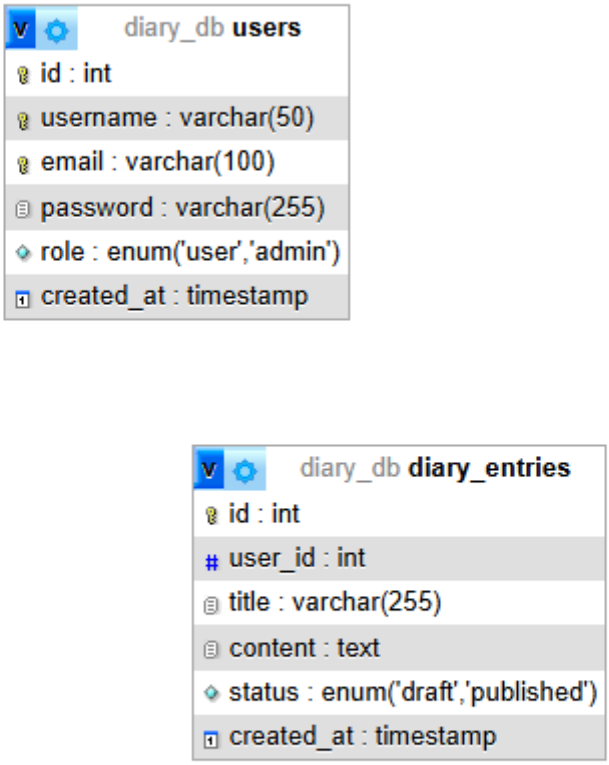

Рис. 2.2. Діаграма Бази Даних

2.4 Реалізація архітектурного ядра вебдодатку

Основою вебдодатку є власна реалізація архітектури MVC. Система побудована на принципі єдиної точки входу, де всі запити обробляються головним файлом конфігурації та передаються до ядра програми.

1. Маршрутизація core/Router.php

Для обробки запитів та динамічного виклику компонентів системи використовується клас Router. Він виконує розбір URL-адреси, визначає

необхідний контролер та метод, а також передає параметри запиту для подальшої обробки бізнес-логіки.

```
public function __construct() {
        $url = $this->parseUrl();

        if (isset($url[0])) {
            $controllerName = ucfirst($url[0]) . 'Controller';
            $file = "../app/Controllers/" . $controllerName . ".php";

            if (file_exists($file)) {
                $this->controller = 'App\Controllers\\' . $controllerName;
                unset($url[0]);
            }
        }
        $this->controller = new $this->controller;

        if (isset($url[1])) {
            if (method_exists($this->controller, $url[1])) {
                $this->method = $url[1];
                unset($url[1]);
            }
        }

        $this->params = $url ? array_values($url) : [];

        // ЗАПУСК: Викликаємо метод контролера з параметрами
        call_user_func_array([$this->controller, $this->method], $this->params);
    }

    private function parseUrl() {
        if (isset($_GET['url'])) {
            return explode('/', filter_var(rtrim($_GET['url'], '/'),
FILTER_SANITIZE_URL));
        }
        return [];
    }
```

2. Базовий контролер core/Controller.php

Усі контролери додатку успадковують базовий клас Controller. Його головне завдання - забезпечити передачу даних від бізнес-логіки до шаблонів інтерфейсу та виконати рендеринг відповідних представлень Views.

```
namespace Core;

class Controller {
    protected function render($view, $data = []) {
```

```
        extract($data);

        $viewPath = "../app/Views/" . $view . ".php";

        if (file_exists($viewPath)) {
            require_once $viewPath;
        } else {
            die("Помилка: View '$view' не знайдено за шляхом $viewPath");
        }
    }
}
```

3. Робота з базою даних core/Database.php

Для взаємодії з сервером бази даних MySQL обрано розширення PDO PHP Data Objects. Клас Database реалізований за допомогою патерну проєктування Singleton, що забезпечує створення лише одного екземпляра класу та гарантує наявність єдиного активного підключення до БД протягом усього життєвого циклу запиту.

```
namespace Core;

use PDO;
use PDOException;

class Database {
    private static $instance = null;
    private $connection;

    private function __construct() {
        try {
            $dsn = "mysql:host=" . DB_HOST . ";dbname=" . DB_NAME .
";charset=utf8mb4";
            $this->connection = new PDO($dsn, DB_USER, DB_PASS, [
                PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC
            ]);
        } catch (PDOException $e) {
            die("Помилка підключення до БД: " . $e->getMessage());
        }
    }

    public static function getInstance() {
        if (self::$instance === null) {
            self::$instance = new self();
        }
        return self::$instance;
```

```
    }

    public function getConnection() {
        return $this->connection;
    }
}
```

4. Модель даних app/models/DiaryEntry.php

Моделі в системі відповідають за безпосередню роботу з таблицями бази даних. Клас DiaryEntry реалізує повний цикл CRUD-операцій створення, читання, оновлення, видалення для записів щоденника. Для забезпечення безпеки даних використовуються виключно підготовлені запити Prepared Statements, що повністю нівелює загрозу SQL-ін'єкцій.

```
namespace App\Models;

use Core\Model;
use PDO;

class DiaryEntry extends Model {

    public function create($data) {
        $sql = "INSERT INTO diary_entries (user_id, title, content) VALUES
(:user_id, :title, :content)";
        $stmt = $this->db->prepare($sql);

        return $stmt->execute([
            ':user_id' => $_SESSION['user_id'],
            ':title'   => $data['title'],
            ':content' => $data['content']
        ]);
    }

    public function getAllByUserId($userId) {
        $sql = "SELECT * FROM diary_entries WHERE user_id = :user_id ORDER BY
created_at DESC";
        $stmt = $this->db->prepare($sql);
        $stmt->execute([':user_id' => $userId]);
        return $stmt->fetchAll();
    }

    public function getById($id, $userId) {
        $sql = "SELECT * FROM diary_entries WHERE id = :id AND user_id = :user_id";
        $stmt = $this->db->prepare($sql);
        $stmt->execute([':id' => $id, ':user_id' => $userId]);
        return $stmt->fetch();
```

```
    }

    public function update($id, $userId, $data) {
        $sql = "UPDATE diary_entries SET title = :title, content = :content WHERE
id = :id AND user_id = :user_id";
        $stmt = $this->db->prepare($sql);
        return $stmt->execute([
            ':id'      => $id,
            ':user_id' => $userId,
            ':title'   => $data['title'],
            ':content' => $data['content']
        ]);
    }

    public function delete($id, $userId) {
        $sql = "DELETE FROM diary_entries WHERE id = :id AND user_id = :user_id";
        $stmt = $this->db->prepare($sql);
        return $stmt->execute([':id' => $id, ':user_id' => $userId]);
    }
}
```

Висновки до другого розділу

У другому розділі проведено проєктування та програмну реалізацію архітектури вебзастосунку Diary. На основі патерну MVC розроблено власне програмне ядро, що забезпечує чіткий поділ логіки, даних та інтерфейсу.

Ключові результати розділу:

- Реалізовано систему маршрутизації на основі парсингу URL-адрес для динамічного виклику контролерів.
- Створено безпечний механізм взаємодії з базою даних MySQL через об'єкт PDO з використанням патерну Singleton.
- Розроблено модель DiaryEntry, яка реалізує повний цикл CRUD-операцій із обов’язковою перевіркою прав доступу користувача на рівні SQL-запитів.
- Впроваджено систему авторизації на основі сесій, що гарантує ізоляцію та конфіденційність персональних нотаток.

- Обрана архітектура забезпечує високу швидкість роботи додатка, захист від SQL-ін’єкцій та легкість у подальшому масштабуванні функціоналу.

РОЗДІЛ 3. ОПИС РОБОТИ З ПРОГРАМНИМ ДОДАТКОМ ТА ЙОГО ТЕСТУВАННЯ

3.1. Опис роботи з програмним додатком

Вебзастосунок Diary має інтуїтивно зрозумілий графічний інтерфейс, виконаний у мінімалістичному стилі. Завдяки модульній системі CSS, описаній у розділі 2, інтерфейс забезпечує комфортну роботу користувача як на стаціонарних комп'ютерах, так і на мобільних пристроях.

Процес роботи з додатком починається з головної сторінки landing page. На цій сторінці неавторизованому відвідувачу пропонується ознайомитися з призначенням сервісу та перейти до авторизації або реєстрації нового акаунта.

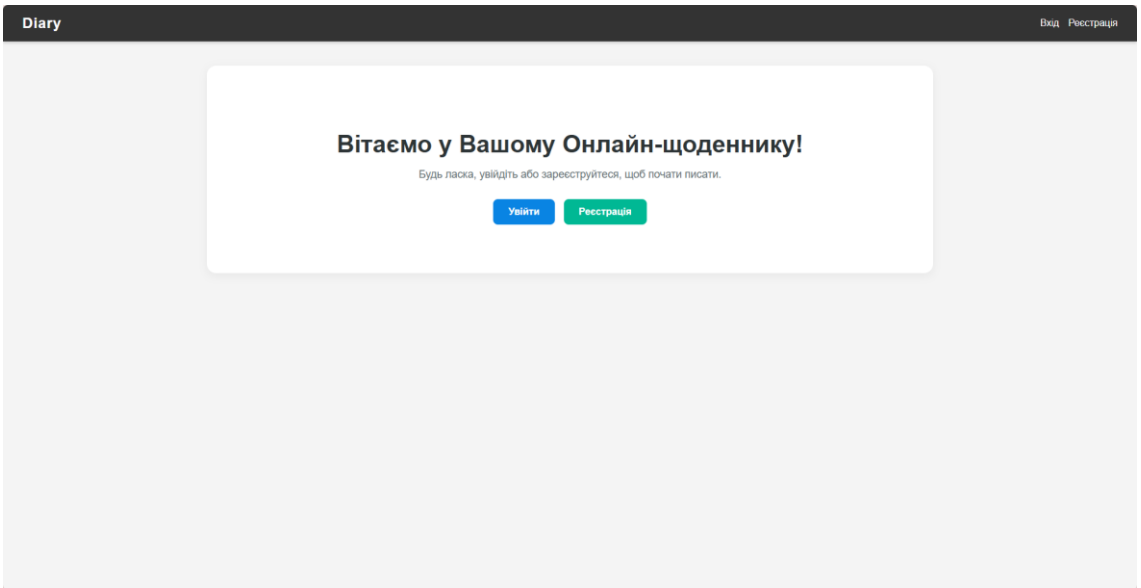

Рис. 3.1. Головна сторінка вебдодатка Diary для неавторизованого користувача.

При натисканні кнопки Реєстрація, або відповідного пункту в навігаційному меню, відкривається екранна форма для створення нового облікового запису. Користувач має ввести унікальний логін, діючу електронну пошту та придумати пароль. Система автоматично перевіряє унікальність Email та складність пароля перед збереженням даних у таблицю users.

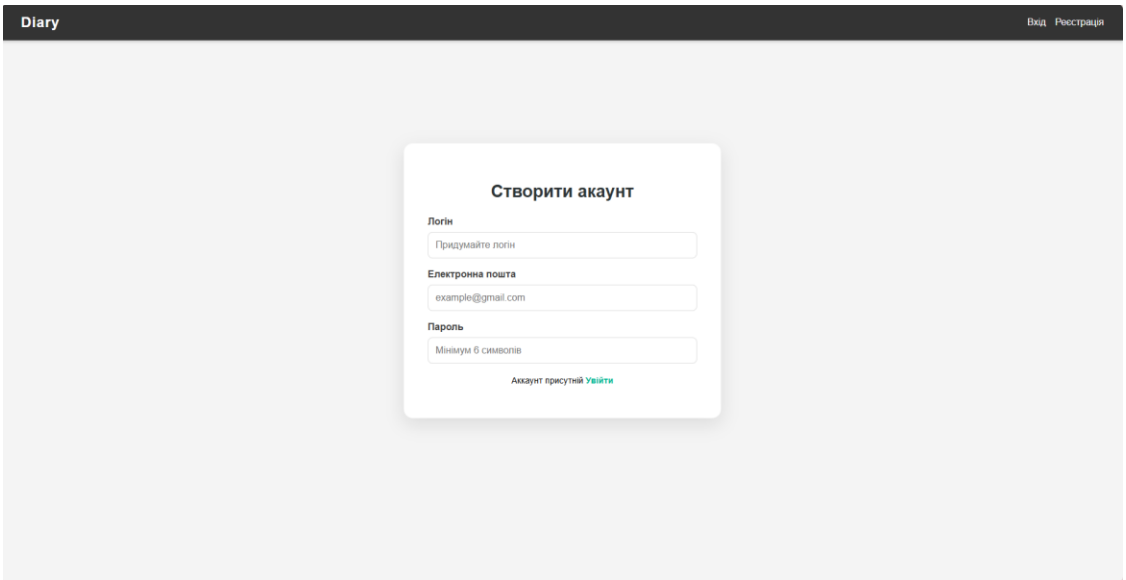

Рис. 3.2. Сторінка реєстрації нового користувача

Для входу в систему використовується форма авторизації. Користувач вводить Email та пароль, вказані при реєстрації. Після успішної перевірки даних яка включає password_verify, ініціалізується сесія, і користувач перенаправляється до панелі керування нотатками.

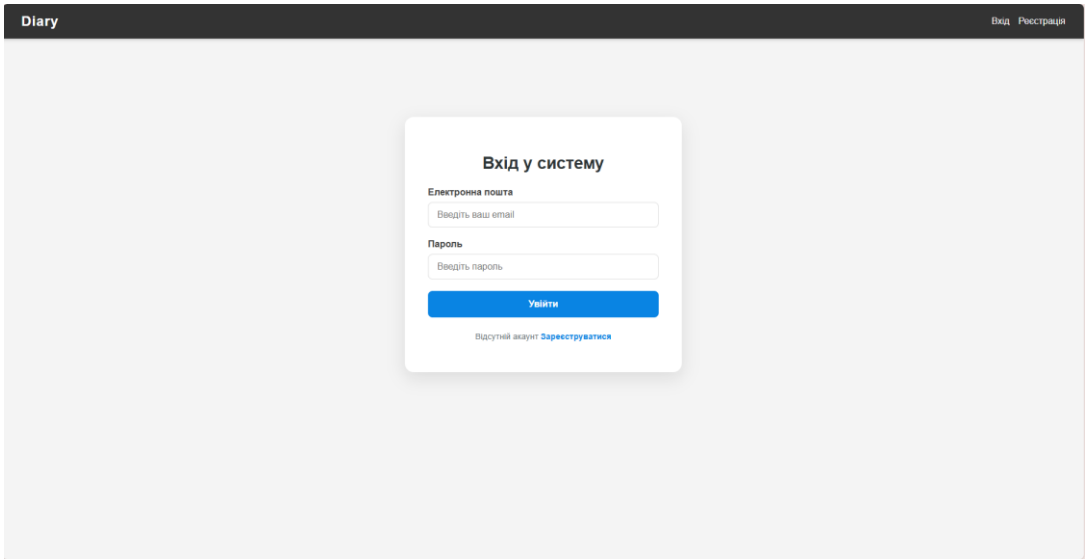

Рис. 3.3. Сторінка входу в систему

Основна робоча область залогіненого користувача - це сторінка Мої записи. Вона виконана у вигляді сітки карток, де кожна картка представляє окрему нотатку. Кожна картка містить заголовок, фрагмент тексту, дату та час створення. У верхній частині сторінки розташована кнопка + Новий запис.

Користувач також має можливість відредагувати або видалити будь-яку нотатку безпосередньо з цього екрана.

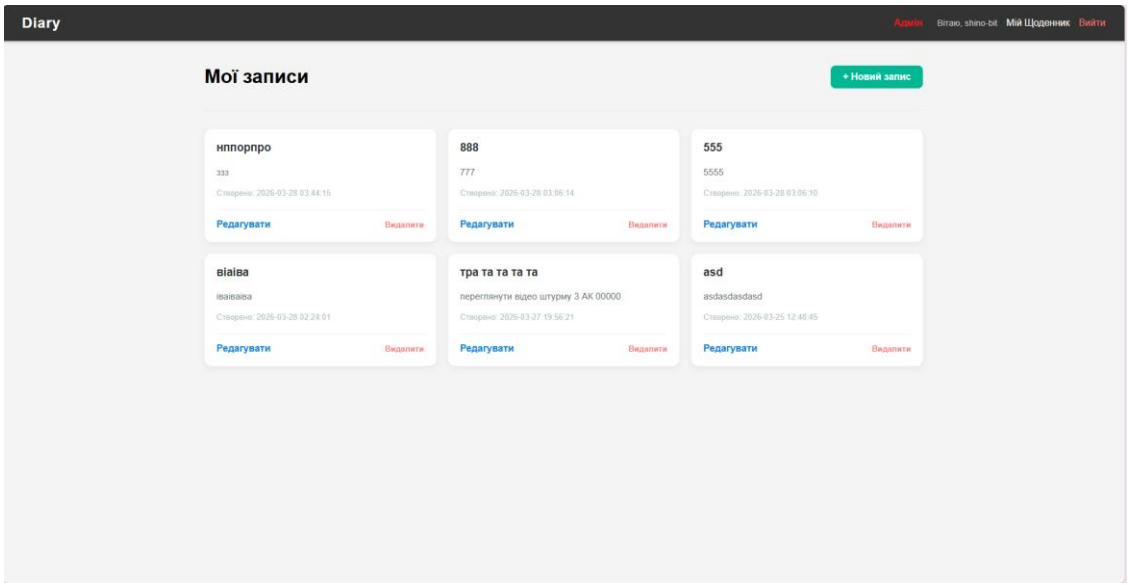

Рис. 3.4. Основний інтерфейс панелі користувача список нотаток

Натискання кнопки + Новий запис відкриває форму створення. Вона містить лише два поля: Заголовок та Зміст нотатки. Після заповнення полів та натискання кнопки Зберегти у щоденник, дані записуються в базу даних і користувач повертається до загального списку.

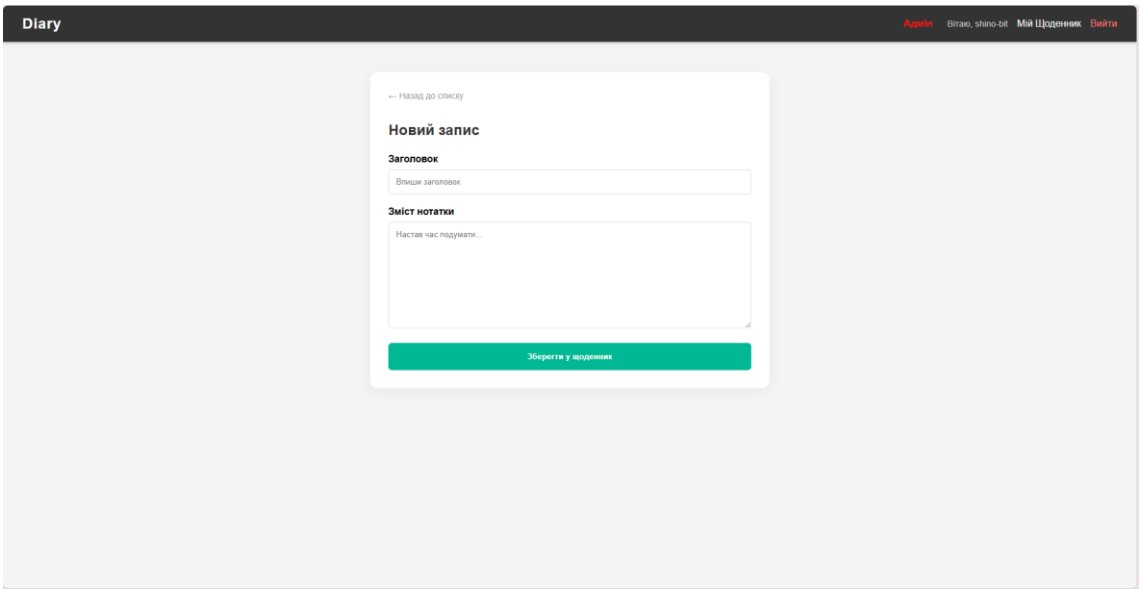

Рис. 3.5. Форма створення нової нотатки

Інтерфейс редагування нотатки ідентичний формі створення, але поля вже заповнені поточною інформацією з бази даних. Користувач може внести

зміни та натиснути Зберегти зміни. Також передбачена кнопка Назад для скасування редагування.

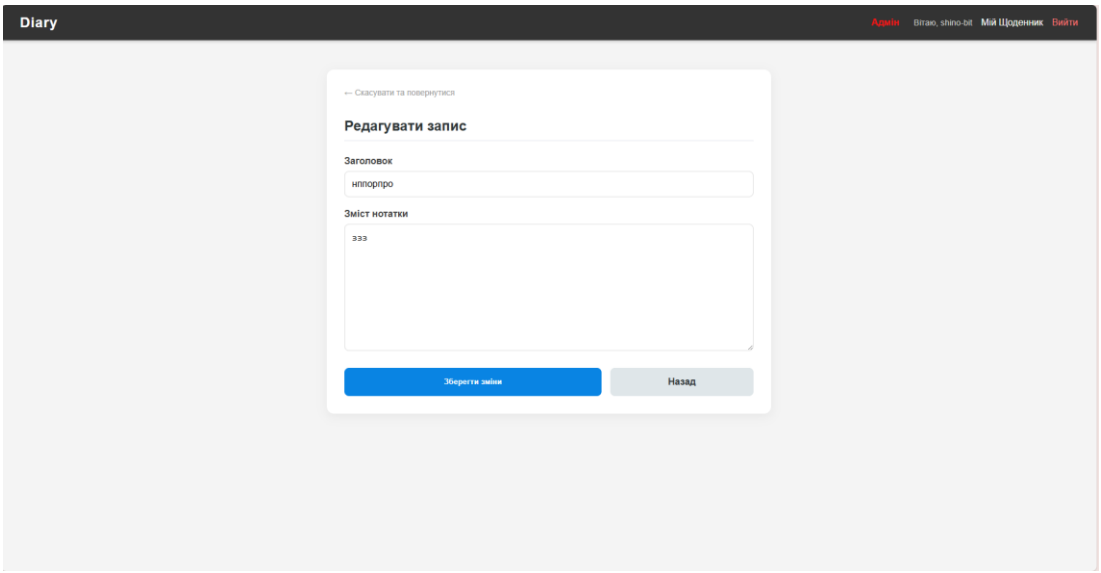

Рис. 3.6. Інтерфейс редагування існуючої нотатки

Для запобігання випадковому видаленню даних, при натисканні кнопки Видалити на картці нотатки, система видає модальне вікно JavaScript confirm з проханням підтвердити дію. Тільки після підтвердження відбувається видалення запису з бази даних MySQL.

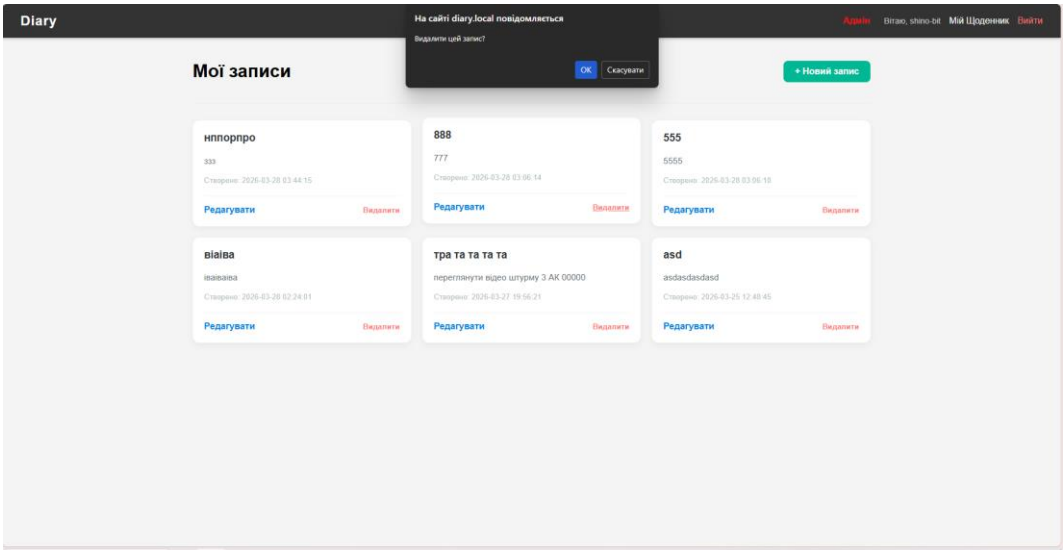

Рис. 3.7. Запит на підтвердження видалення запису

Якщо користувач має права адміністратора що визначається полем role = 'admin' у таблиці users, у навігаційному меню з'являється додатковий пункт. Натискання на нього відкриває Панель адміністратора, де відображено таблицю

всіх зареєстрованих користувачів системи з їхніми ID, логінами та ролями. Адміністратор має можливість видалити будь-якого користувача за допомогою відповідної кнопки в таблиці.

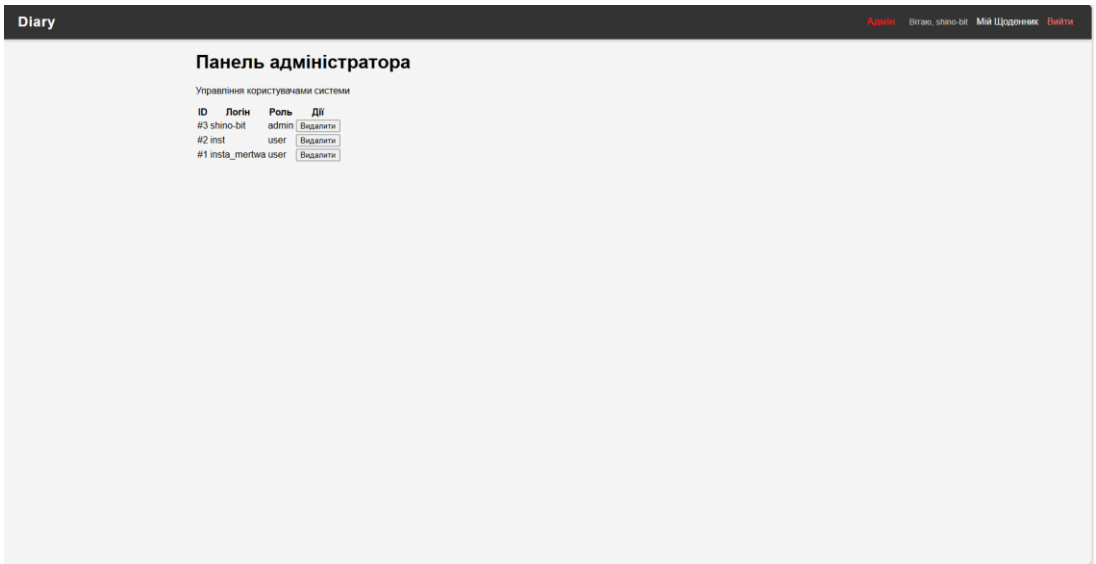

Рис. 3.8. Інтерфейс панелі адміністратора список користувачів системи

3.2. Тестування роботи програмного забезпечення

Після завершення розробки основної логіки вебдодатку Diary було проведено комплексне ручне тестування ключових модулів системи. У ході перевірки було виявлено низку технічних дефектів, які могли вплинути на безпеку даних користувачів або коректність відображення інтерфейсу. Нижче наведено опис виявлених проблем та способи їх усунення:

1. Некоректна обробка порожнього змісту нотатки

Під час тестування форми створення запису було виявлено, що при спробі зберегти нотатку з порожнім полем Зміст, база даних повертала помилку або записувала пустий рядок, що виглядало некоректно в інтерфейсі списку записів.

Вирішення: На рівні моделі DiaryEntry було впроваджено серверу валідацію: якщо поле змісту порожнє, система автоматично присвоює йому значення NULL або повертає повідомлення про помилку, не дозволяючи створити пустий запис.

2. **Вразливість до SQL-ін’єкцій у параметрах ідентифікації**

Було виявлено, що в ранніх версіях методів getById та delete параметри id нотатки могли потрапляти в запит без належної типізації. Це створювало потенційну загрозу маніпуляції базою даних через адресний рядок браузера.

Вирішення: Усі SQL-запити в моделях були переведені на використання Prepared Statements підготовлених запитів через PDO. Крім того, додано обов'язкове приведення типу (int)$id для всіх вхідних ідентифікаторів.

3. **Збій редиректу після авторизації через складні URL**

Параметр, що відповідає за перенаправлення користувача на попередню сторінку після входу в систему, некоректно обробляв URL-адреси, що містили власні параметри наприклад, при спробі відкрити конкретну нотатку без авторизації.

Вирішення: Для надійної передачі шляху було впроваджено механізм base64_encode() для кодування адреси в параметрі редиректу та base64_decode() для її відновлення після успішного входу. Це дозволило уникнути конфліктів символів у адресному рядку.

4. **Помилка ініціалізації об’єкта бази даних**

При інтенсивному тестуванні виникала помилка повторного створення підключення до БД в різних частинах коду, що призводило до вичерпання ліміту з’єднань.

Вирішення: Остаточне впровадження патерну Singleton у класі Database як описано в Розділі 2) вирішило цю проблему. Тепер метод getInstance() гарантує використання одного й того самого об'єкта підключення протягом усього ч

Усі виявлені під час тестування недоліки були оперативно усунені. Проведені заходи дозволили значно підвищити надійність системи, забезпечити конфіденційність записів користувачів та стабільну роботу вебдодатку в умовах реальної експлуатації.асу роботи скрипта.

Висновок третього розділу

У третьому розділі було детально розглянуто практичні аспекти функціонування вебдодатка Diary. Описано повний цикл взаємодії користувача з системою: від реєстрації та авторизації до безпосереднього керування персональними нотатками створення, перегляд, редагування та видалення. Окрему увагу приділено інтерфейсу адміністратора, що дозволяє ефективно керувати обліковими записами користувачів.

Проведене ручне тестування основних модулів дозволило виявити та оперативно усунути низку технічних недоліків.

ВИСНОВКИ

У курсовій роботі виконано повний цикл розробки вебдодатка Diary - від аналізу предметної області та формування технічних вимог до етапу впровадження й тестування готового програмного рішення. На аналітичному етапі було досліджено сучасні підходи до створення сервісів персональних нотаток, проаналізовано існуючі рішення для зберігання конфіденційної інформації та визначено критичний набір функцій, необхідних для зручної та безпечної роботи користувача в онлайн-середовищі.

У процесі проєктування було розроблено загальний алгоритм взаємодії користувачів із системою та створено власне архітектурне ядро на основі патерну MVC. Реалізований фреймворк включає систему динамічної маршрутизації, базові контролери та механізм рендерингу представлень із передачею даних через екстракцію масивів. Структура бази даних MySQL була спроєктована таким чином, щоб забезпечити гнучке відображення сутностей користувачів та їхніх персональних записів, гарантуючи цілісність даних та можливість подальшого масштабування системи.

У підсумку створено стабільний, зручний та захищений вебсервіс для ведення особистого щоденника. Розроблений застосунок повністю відповідає поставленим завданням і готовий до практичного використання як персональний інструмент для нотаток. Завдяки модульній архітектурі та чіткому поділу відповідальності між компонентами системи, проєкт має високий потенціал для подальшого розвитку, зокрема впровадження категорій записів, пошуку за ключовими словами або розширених налаштувань профілю користувача.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. PHP: The Right Way - https://phptherightway.com/
2. Refactoring.Guru: Singleton in PHP https://refactoring.guru/design-patterns/singleton/php/example
3. MDN Web Docs: HTTP Headers (Location) - https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Location
4. OWASP:SQLInjectionPreventionCheatSheet - https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
5. SitePoint: The MVC Pattern and PHP. https://www.sitepoint.com/the-mvc-pattern-and-php/
6. MySQL Reference Manual: INSERT/UPDATE/DELETE Statements https://dev.mysql.com/doc/refman/8.0/en/sql-statements.html
7. PHP Manual: Sessions https://www.php.net/manual/en/book.session.php
8. Composer Documentation https://getcomposer.org/doc/
9. W3Schools: PHP Filter Variable https://www.w3schools.com/php/func_filter_var.asp

ДОДАТКИ

Файл AdminController.php:

```php
<?php
namespace App\Controllers;

use Core\Controller;
use App\Models\User;
use App\Models\DiaryEntry;

class AdminController extends Controller {

    public function __construct() {
        // Жорстка перевірка прав доступу
        if (!isset($_SESSION['user_id']) || $_SESSION['role'] !== 'admin') {
            header('Location: /'); // Викидаємо звичайних юзерів на головну
            exit;
        }
    }

    public function index() {
        $userModel = new User();
        $users = $userModel->getAll();

        $this->render('admin/index', [
            'title' => 'Панель адміністратора',
            'users' => $users
        ]);
    }
}
```

Файл AuthController.php:

```php
<?php
namespace App\Controllers;

use Core\Controller;
use App\Models\User;

class AuthController extends Controller {

    /**
     * Дефолтний метод, який викликає роутер, якщо не вказано дію.
     */
    public function index() {
        $this->login();
    }

    public function register() {
```

```
        $this->render('auth/register', ['title' => 'Реєстрація']);
    }

    public function store() {
        if ($_SERVER['REQUEST_METHOD'] === 'POST') {
            $userModel = new User();

            $userData = [
                'username' => $_POST['username'],
                'email'    => $_POST['email'],
                'password' => $_POST['password']
            ];

            if ($userModel->create($userData)) {
                header('Location: /auth/login');
                exit;
            } else {
                // Виводимо помилку в інтерфейс, а не просто текстом
                $this->render('auth/register', [
                    'title' => 'Реєстрація',
                    'error' => 'Помилка при реєстрації. Можливо, цей Email вже
зайнятий.'
                ]);
            }
        }
    }

    public function login() {
        $this->render('auth/login', ['title' => 'Вхід']);
    }

    /**
     * Перейменовуємо 'auth' на 'authenticate', щоб збігалося з URL
/auth/authenticate
     */
    public function authenticate() {
        if ($_SERVER['REQUEST_METHOD'] === 'POST') {
            $userModel = new User();
            $user = $userModel->findByEmail($_POST['email']);

            if ($user && password_verify($_POST['password'], $user['password'])) {
                $_SESSION['user_id'] = $user['id'];
                $_SESSION['username'] = $user['username'];
                $_SESSION['role'] = $user['role'];

                header('Location: /diary/index');
                exit;
            } else {
                // Виводимо помилку через рендер у наш CSS-блок
```

```
                $this->render('auth/login', [
                    'title' => 'Вхід',
                    'error' => 'Невірний email або пароль!'
                ]);
            }
        }
    }

    public function logout() {
        $_SESSION = [];

        if (isset($_COOKIE[session_name()])) {
            setcookie(session_name(), '', time() - 3600, '/');
        }

        session_destroy();

        header('Location: /');
        exit;
    }
}
```

Файл DiaryController.php:

```
<?php
namespace App\Controllers;

use Core\Controller;
use App\Models\DiaryEntry;

class DiaryController extends Controller {

    public function __construct() {
        if (!isset($_SESSION['user_id'])) {
            header('Location: /auth/login');
            exit;
        }
    }

    public function index() {
        $diaryModel = new DiaryEntry();
        $entries = $diaryModel->getAllByUserId($_SESSION['user_id']);

        $this->render('diary/index', [
            'title' => 'Мій Щоденник',
            'entries' => $entries
        ]);
```

```
    }

    public function create() {
        $this->render('diary/create', ['title' => 'Новий запис']);
    }

    public function store() {
        if ($_SERVER['REQUEST_METHOD'] === 'POST') {
            $diaryModel = new DiaryEntry();

            if ($diaryModel->create($_POST)) {
                header('Location: /diary/index');
                exit;
            }
        }
    }

    public function edit($id) {
        $diaryModel = new DiaryEntry();
        $entry = $diaryModel->getById($id, $_SESSION['user_id']);

        if (!$entry) {
            die("Запис не знайдено або у вас немає прав на його редагування.");
        }

        $this->render('diary/edit', ['title' => 'Редагувати запис', 'entry' =>
$entry]);
    }

    public function update($id) {
        if ($_SERVER['REQUEST_METHOD'] === 'POST') {
            $diaryModel = new DiaryEntry();
            if ($diaryModel->update($id, $_SESSION['user_id'], $_POST)) {
                header('Location: /diary/index');
                exit;
            }
        }
    }

    public function destroy($id) {
        $diaryModel = new DiaryEntry();
        if ($diaryModel->delete($id, $_SESSION['user_id'])) {
            header('Location: /diary/index');
            exit;
        }
    }
}
```

Файл HomeController.php:

```php
<?php
namespace App\Controllers;

use Core\Controller;

class HomeController extends Controller {
    public function index() {
    $this->render('home', [
        'title' => 'Головна сторінка'
    ]);
}
}
```

Файл DiaryEntry.php:

```php
<?php
namespace App\Models;

use Core\Model;
use PDO;

class DiaryEntry extends Model {

    public function create($data) {
        $sql = "INSERT INTO diary_entries (user_id, title, content) VALUES
(:user_id, :title, :content)";
        $stmt = $this->db->prepare($sql);

        return $stmt->execute([
            ':user_id' => $_SESSION['user_id'],
            ':title'   => $data['title'],
            ':content' => $data['content']
        ]);
    }

    public function getAllByUserId($userId) {
        $sql = "SELECT * FROM diary_entries WHERE user_id = :user_id ORDER BY
created_at DESC";
        $stmt = $this->db->prepare($sql);
        $stmt->execute([':user_id' => $userId]);
        return $stmt->fetchAll();
    }

    public function getById($id, $userId) {
        $sql = "SELECT * FROM diary_entries WHERE id = :id AND user_id = :user_id";
        $stmt = $this->db->prepare($sql);
        $stmt->execute([':id' => $id, ':user_id' => $userId]);
```

```
        return $stmt->fetch();
    }

    public function update($id, $userId, $data) {
        $sql = "UPDATE diary_entries SET title = :title, content = :content WHERE
id = :id AND user_id = :user_id";
        $stmt = $this->db->prepare($sql);
        return $stmt->execute([
            ':id'      => $id,
            ':user_id' => $userId,
            ':title'   => $data['title'],
            ':content' => $data['content']
        ]);
    }

    public function delete($id, $userId) {
        $sql = "DELETE FROM diary_entries WHERE id = :id AND user_id = :user_id";
        $stmt = $this->db->prepare($sql);
        return $stmt->execute([':id' => $id, ':user_id' => $userId]);
    }
}
```

Файл User.php:

```
<?php
namespace App\Models;

use Core\Model;
use PDO;

class User extends Model {
    public function create($data) {
        $sql = "INSERT INTO users (username, email, password, role) VALUES
(:username, :email, :password, :role)";
        $stmt = $this->db->prepare($sql);

        $data['password'] = password_hash($data['password'], PASSWORD_DEFAULT);

        return $stmt->execute([
            ':username' => $data['username'],
            ':email'    => $data['email'],
            ':password' => $data['password'],
            ':role'     => $data['role'] ?? 'user'
        ]);
    }

    public function findByEmail($email) {
```

```
        $sql = "SELECT * FROM users WHERE email = :email";
        $stmt = $this->db->prepare($sql);
        $stmt->execute([':email' => $email]);
        return $stmt->fetch();
    }

    public function getAll() {
        $sql = "SELECT id, username, email, role, created_at FROM users ORDER BY
created_at DESC";
        $stmt = $this->db->query($sql);
        return $stmt->fetchAll();
    }
}
```

Файл index.php:

```
<?php require_once __DIR__ . '/../header.php'; ?>

<div class="admin-container">
    <div class="admin-header">
        <h1>Панель адміністратора</h1>
        <p class="admin-subtitle">Управління користувачами системи</p>
    </div>

    <div class="admin-card">
        <table class="admin-table">
            <thead>
                <tr>
                    <th>ID</th>
                    <th>Логін</th>
                    <th>Роль</th>
                    <th>Дії</th>
                </tr>
            </thead>
            <tbody>
                <?php if (!empty($users)): ?>
                    <?php foreach ($users as $user): ?>
                        <tr>
                            <td>#<?php echo $user['id']; ?></td>
                            <td class="user-name"><?php echo
htmlspecialchars($user['username']); ?></td>
                            <td>
                                <span class="role-badge <?php echo ($user['role']
=== 'admin') ? 'role-admin' : 'role-user'; ?>">
                                    <?php echo htmlspecialchars($user['role']); ?>
                                </span>
                            </td>
```

```
                            <td class="admin-actions">
                                <form action="/admin/delete/<?php echo $user['id'];
?>" method="POST" onsubmit="return confirm('Видалити цього користувача?')">
                                    <button type="submit" class="btn-delete-
user">Видалити</button>
                                </form>
                            </td>
                        </tr>
                    <?php endforeach; ?>
                <?php else: ?>
                    <tr>
                        <td colspan="4" style="text-align: center; padding:
20px;">Користувачів не знайдено</td>
                    </tr>
                <?php endif; ?>
            </tbody>
        </table>
    </div>
</div>

</div> </body>
</html>
```

Файл login.php:

```
<?php require_once __DIR__ . '/../header.php'; ?>

<div class="auth-container">
    <div class="auth-card">
        <h2 class="auth-title">Вхід у систему</h2>

        <?php if (isset($error)): ?>
            <div class="auth-error">
                <?php echo $error; ?>
            </div>
        <?php endif; ?>

        <form action="/auth/authenticate" method="POST" class="auth-form">
            <div class="form-group">
                <label for="email">Електронна пошта</label>
                <input type="email" id="email" name="email" placeholder="Введіть
ваш email" required>
            </div>

            <div class="form-group">
                <label for="password">Пароль</label>
```

```
                <input type="password" id="password" name="password"
placeholder="Введіть пароль" required>
            </div>

            <button type="submit" class="btn-login">Увійти</button>
        </form>

        <div class="auth-footer">
            <p>Відсутній акаунт <a href="/auth/register">Зареєструватися</a></p>
        </div>
    </div>
</div>

</div> </body>
</html>
```

Файл register.php:

```
<?php require_once __DIR__ . '/../header.php'; ?>

<div class="auth-container">
    <div class="auth-card register-card">
        <h2 class="auth-title">Створити акаунт</h2>

        <?php if (isset($error)): ?>
            <div class="auth-error">
                <?php echo $error; ?>
            </div>
        <?php endif; ?>

        <form action="/auth/store" method="POST" class="auth-form">
        <div class="form-group">
            <label for="username">Логін</label>
            <input type="text" id="username" name="username"
placeholder="Придумайте логін" required>
        </div>

        <div class="form-group">
            <label for="email">Електронна пошта</label>
            <input type="email" id="email" name="email"
placeholder="example@gmail.com" required>
        </div>

        <div class="form-group">
            <label for="password">Пароль</label>
            <input type="password" id="password" name="password"
placeholder="Мінімум 6 символів" required>
```

```
        </div>

        </form>
        <div class="auth-footer">
            <p>Аккаунт присутній <a href="/auth/login">Увійти</a></p>
        </div>
    </div>
</div>

</div> </body>
</html>
```

Файл create.php:

```
<?php require_once __DIR__ . '/../header.php'; ?>

<div class="diary-container">
    <div class="diary-card">
        <a href="/diary/index" class="back-link">← Назад до списку</a>
        <h2 class="diary-title">Новий запис</h2>

        <form action="/diary/store" method="POST" class="diary-form">
            <div class="form-group">
                <label>Заголовок</label>
                <input type="text" name="title" placeholder="Впиши заголовок"
required>
            </div>

            <div class="form-group">
                <label>Зміст нотатки</label>
                <textarea name="content" placeholder="Настав час подумати..."
rows="10" required></textarea>
            </div>

            <button type="submit" class="btn btn-save">
                Зберегти у щоденник
            </button>
        </form>
    </div>
</div>

</div>
</body>
</html>
```

Файл edit.php:

```
<?php require_once __DIR__ . '/../header.php'; ?>

<div class="diary-container">
    <div class="diary-card">
        <a href="/diary/index" class="back-link">← Скасувати та повернутися</a>
        <h2 class="diary-title"> Редагувати запис</h2>

        <form action="/diary/update/<?php echo $entry['id']; ?>" method="POST"
class="diary-form">
            <div class="form-group">
                <label>Заголовок</label>
                <input type="text" name="title" value="<?php echo
htmlspecialchars($entry['title']); ?>" required>
            </div>

            <div class="form-group">
                <label>Зміст нотатки</label>
                <textarea name="content" rows="10" required><?php echo
htmlspecialchars($entry['content']); ?></textarea>
            </div>

            <div class="form-actions">
                <button type="submit" class="btn-update">Зберегти зміни</button>
                <a href="/diary/index" class="btn-cancel">Назад</a>
            </div>
        </form>
    </div>
</div>

</div> </body>
</html>
```

Файл index.php:

```
<?php require_once __DIR__ . '/../header.php'; ?>

<div class="diary-header">
    <h1>Мої записи</h1>
    <a href="/diary/create" class="btn btn-create">+ Новий запис</a>
</div>

<?php if (empty($entries)): ?>
    <div class="empty-state">
        <p>У вас ще немає жодного запису. Настав час щось написати!</p>
    </div>
<?php else: ?>
```

```
    <div class="diary-grid">
        <?php foreach ($entries as $entry): ?>
            <div class="diary-card">
                <div class="card-content">
                    <h3><?php echo htmlspecialchars($entry['title']); ?></h3>
                    <p><?php echo
mb_strimwidth(htmlspecialchars($entry['content']), 0, 150, "..."); ?></p>
                    <small>Створено: <?php echo $entry['created_at']; ?></small>
                </div>

                <div class="card-actions">
                    <a href="/diary/edit/<?php echo $entry['id']; ?>" class="btn-
edit">Редагувати</a>
                    <form action="/diary/delete/<?php echo $entry['id']; ?>"
method="POST" onsubmit="return confirm('Видалити цей запис?')">
                        <button type="submit" class="btn-delete">Видалити</button>
                    </form>
                </div>
            </div>
        <?php endforeach; ?>
    </div>
<?php endif; ?>

</div> </body>
</html>
```

Файл header.php:

```
<!DOCTYPE html>
<html lang="uk">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title><?php echo $title ?? 'Щоденник'; ?></title>

    <link rel="stylesheet" href="/css/header.css">

    <?php
        $url = isset($_GET['url']) ? explode('/', rtrim($_GET['url'], '/')) :
['home'];

        // Визначаємо пріоритет:
        // 1. Якщо є друга частина URL (наприклад, 'create'), шукаємо 'create.css'
        // 2. Якщо немає, беремо першу частину (наприклад, 'diary') і шукаємо
'diary.css'
        $styleName = (isset($url[1]) && !empty($url[1])) ? $url[1] : $url[0];
```

```
        // Перевіряємо наявність файлу в папці public/css/
        if (file_exists("css/{$styleName}.css")): ?>
            <link rel="stylesheet" href="/css/<?php echo $styleName; ?>.css">
    <?php endif; ?>
</head>
<body>

<header>
    <div class="header-logo">
        <strong>Diary</strong>
    </div>
    <nav>
        <?php if (isset($_SESSION['user_id'])): ?>
            <?php if ($_SESSION['role'] === 'admin'): ?>
                <a href="/admin/index" class="nav-link-admin">Адмін</a>
            <?php endif; ?>

            <span class="user-welcome">Вітаю, <?php echo $_SESSION['username'];
?></span>
            <a href="/diary/index" class="nav-link">Мій Щоденник</a>
            <a href="/auth/logout" class="nav-link-logout">Вийти</a>
        <?php else: ?>
            <a href="/auth/login" class="nav-link">Вхід</a>
            <a href="/auth/register" class="nav-link">Реєстрація</a>
        <?php endif; ?>
    </nav>
</header>

<div class="container">
```

Файл home.php:

```
<?php require_once __DIR__ . '/header.php'; ?>

<div class="home-hero">
    <?php if (isset($_SESSION['user_id'])): ?>
        <h1>Вітаємо, <?php echo $_SESSION['username']; ?>! 👋</h1>
        <p class="status-badge">Твій статус: <strong><?php echo $_SESSION['role'];
?></strong></p>
        <div class="home-actions">
            <a href="/diary/index" class="btn btn-primary">Перейти до щоденника</a>
        </div>
    <?php else: ?>
        <h1>Вітаємо у Вашому Онлайн-щоденнику!</h1>
        <p class="home-subtitle">Будь ласка, увійдіть або зареєструйтеся, щоб
почати писати.</p>
        <div class="home-actions">
```

```
            <a href="/auth/login" class="btn btn-primary">Увійти</a>
            <a href="/auth/register" class="btn btn-success">Реєстрація</a>
        </div>
    <?php endif; ?>
</div>

</div> </body>
</html>
```

Файл config.php:

```
<?php
define('DB_HOST', 'localhost');
define('DB_NAME', 'diary_db');
define('DB_USER', 'root');
define('DB_PASS', '12345');

define('BASE_URL', 'http://diary.local');
```

Файл Controller.php:

```
<?php
namespace Core;

class Controller {
    protected function render($view, $data = []) {
        extract($data);

        $viewPath = "../app/Views/" . $view . ".php";

        if (file_exists($viewPath)) {
            require_once $viewPath;
        } else {
            die("Помилка: View '$view' не знайдено за шляхом $viewPath");
        }
    }
}
```

Файл Database.php:

```
<?php
namespace Core;
```

```php
use PDO;
use PDOException;

class Database {
    private static $instance = null;
    private $connection;

    private function __construct() {
        try {
            $dsn = "mysql:host=" . DB_HOST . ";dbname=" . DB_NAME .
";charset=utf8mb4";
            $this->connection = new PDO($dsn, DB_USER, DB_PASS, [
                PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC
            ]);
        } catch (PDOException $e) {
            die("Помилка підключення до БД: " . $e->getMessage());
        }
    }

    public static function getInstance() {
        if (self::$instance === null) {
            self::$instance = new self();
        }
        return self::$instance;
    }

    public function getConnection() {
        return $this->connection;
    }
}
```

Файл Model.php:

```php
<?php
namespace Core;

abstract class Model {
    protected $db;

    public function __construct() {
        $this->db = Database::getInstance()->getConnection();
    }
}
```

Файл Router.php:

```php
<?php
namespace Core;

class Router {
    protected $controller = 'App\Controllers\HomeController';
    protected $method = 'index';
    protected $params = [];

    public function __construct() {
        $url = $this->parseUrl();

        if (isset($url[0])) {
            $controllerName = ucfirst($url[0]) . 'Controller';
            $file = "../app/Controllers/" . $controllerName . ".php";

            if (file_exists($file)) {
                $this->controller = 'App\Controllers\\' . $controllerName;
                unset($url[0]);
            }
        }
        $this->controller = new $this->controller;

        if (isset($url[1])) {
            if (method_exists($this->controller, $url[1])) {
                $this->method = $url[1];
                unset($url[1]);
            }
        }

        $this->params = $url ? array_values($url) : [];

        // ЗАПУСК: Викликаємо метод контролера з параметрами
        call_user_func_array([$this->controller, $this->method], $this->params);
    }

    private function parseUrl() {
        if (isset($_GET['url'])) {
            return explode('/', filter_var(rtrim($_GET['url'], '/'),
FILTER_SANITIZE_URL));
        }
        return [];
    }
}
```

Файл admin.css:

```
.admin-header {
    margin-bottom: 30px;
    border-bottom: 2px solid #dfe6e9;
    padding-bottom: 15px;
}

.admin-subtitle {
    color: #636e72;
    font-size: 1rem;
}

.admin-card {
    background: #fff;
    padding: 20px;
    border-radius: 12px;
    box-shadow: 0 4px 20px rgba(0,0,0,0.08);
    overflow-x: auto; /* Для мобільних пристроїв */
}

/* Стилізація таблиці */
.admin-table {
    width: 100%;
    border-collapse: collapse;
    text-align: left;
}

.admin-table th {
    background-color: #f8f9fa;
    padding: 15px;
    color: #2d3436;
    font-weight: bold;
    border-bottom: 2px solid #eee;
}

.admin-table td {
    padding: 15px;
    border-bottom: 1px solid #f1f2f6;
    vertical-align: middle;
}

.user-name {
    font-weight: 600;
    color: #2d3436;
}

/* Бейджи ролей */
.role-badge {
    padding: 5px 12px;
    border-radius: 20px;
```

```
    font-size: 0.85rem;
    font-weight: bold;
    text-transform: uppercase;
}

.role-admin {
    background-color: #ffeaa7;
    color: #d63031; /* Червоний текст для адміна */
}

.role-user {
    background-color: #dfe6e9;
    color: #2d3436;
}

.btn-delete-user {
    background: none;
    border: 1px solid #ff7675;
    color: #ff7675;
    padding: 5px 10px;
    border-radius: 5px;
    cursor: pointer;
    font-size: 0.85rem;
    transition: 0.3s;
}

.btn-delete-user:hover {
    background-color: #ff7675;
    color: white;
}
```

Файл create.css:

```
.diary-container {
    display: flex;
    justify-content: center;
    padding: 30px 0;
}

.diary-card {
    background: #fff;
    padding: 30px;
    border-radius: 12px;
    box-shadow: 0 5px 20px rgba(0,0,0,0.05);
    width: 100%;
    max-width: 600px;
}
```

```
.back-link {
    text-decoration: none;
    color: #999;
    font-size: 14px;
    display: inline-block;
    margin-bottom: 15px;
}

.diary-title {
    margin-bottom: 25px;
    color: #333;
}

/* Поля вводу */
.form-group {
    margin-bottom: 20px;
}

.form-group label {
    display: block;
    font-weight: bold;
    margin-bottom: 8px;
}

input[type="text"], textarea {
    width: 100%;
    padding: 12px;
    border: 1px solid #ddd;
    border-radius: 6px;
    box-sizing: border-box; /* Важливо! */
    font-family: inherit;
}

.btn-save {
    background-color: #00b894;
    color: white;
    border: none;
    padding: 15px 20px;
    border-radius: 6px;
    width: 100%;
    font-weight: bold;
    cursor: pointer;
    transition: background 0.2s;
}

.btn-save:hover {
    background-color: #00a383;
}
```

Файл edit.css:

```
.diary-container {
    display: flex;
    justify-content: center;
    padding: 30px 0;
}

.diary-card {
    background: #fff;
    padding: 30px;
    border-radius: 12px;
    box-shadow: 0 5px 20px rgba(0,0,0,0.05);
    width: 100%;
    max-width: 700px;
}

.back-link {
    text-decoration: none;
    color: #999;
    font-size: 14px;
    margin-bottom: 15px;
    display: inline-block;
}

.diary-title {
    margin-bottom: 25px;
    color: #2d3436;
    border-bottom: 2px solid #f1f2f6;
    padding-bottom: 10px;
}

.form-group {
    margin-bottom: 20px;
}

.form-group label {
    display: block;
    font-weight: bold;
    margin-bottom: 8px;
    color: #444;
}

input[type="text"], textarea {
    width: 100%;
    padding: 12px;
    border: 1px solid #ddd;
    border-radius: 8px;
```

```
    box-sizing: border-box;
    font-size: 16px;
}

/* Блок кнопок */
.form-actions {
    display: flex;
    gap: 15px;
    margin-top: 25px;
}

.btn-update {
    background-color: #0984e3;
    color: white;
    border: none;
    padding: 15px 25px;
    border-radius: 8px;
    font-weight: bold;
    cursor: pointer;
    flex: 2;
    transition: background 0.2s;
}

.btn-update:hover {
    background-color: #0a73c5;
}

.btn-cancel {
    background-color: #dfe6e9;
    color: #2d3436;
    text-decoration: none;
    padding: 15px 25px;
    border-radius: 8px;
    font-weight: bold;
    text-align: center;
    flex: 1;
    transition: background 0.2s;
}

.btn-cancel:hover {
    background-color: #b2bec3;
}
```

Файл header.css:

```
body {
    margin: 0;
```

```
    font-family: Arial, sans-serif;
    background-color: #f4f4f4;
}

header {
    background-color: #333;
    color: white;
    padding: 1rem 2rem;
    display: flex;
    justify-content: space-between;
    align-items: center;
    box-shadow: 0 2px 5px rgba(0,0,0,0.2);
}

.header-logo strong {
    font-size: 1.5rem;
    letter-spacing: 1px;
}

nav {
    display: flex;
    align-items: center;
    gap: 15px;
}

nav a {
    text-decoration: none;
    color: white;
    transition: opacity 0.2s;
}

nav a:hover {
    opacity: 0.8;
}

.user-welcome {
    color: #ccc;
    font-size: 0.9rem;
}

.nav-link-admin {
    color: #EE1515 !important;
    font-weight: bold;
    margin-right: 10px;
}

.nav-link-logout {
    color: #ff7675 !important;
}
```

```
.container {
    max-width: 1200px;
    margin: 20px auto;
    padding: 0 15px;
}
```

Файл home.css:

```
.home-hero {
    text-align: center;
    padding: 80px 20px;
    background: #ffffff;
    border-radius: 15px;
    box-shadow: 0 4px 15px rgba(0,0,0,0.05);
    margin-top: 40px;
}

.home-hero h1 {
    font-size: 2.5rem;
    color: #2d3436;
    margin-bottom: 10px;
}

.home-subtitle {
    color: #636e72;
    font-size: 1.1rem;
}

.status-badge {
    display: inline-block;
    background: #f1f2f6;
    padding: 8px 15px;
    border-radius: 20px;
    font-size: 0.9rem;
    color: #2f3542;
    margin-top: 10px;
}

.status-badge strong {
    color: #0984e3;
    text-transform: uppercase;
}

.home-actions {
    margin-top: 30px;
    display: flex;
```

```
    justify-content: center;
    gap: 15px;
}

.btn {
    padding: 12px 25px;
    text-decoration: none;
    border-radius: 8px;
    font-weight: 600;
    transition: transform 0.2s, box-shadow 0.2s;
}

.btn:hover {
    transform: translateY(-2px);
    box-shadow: 0 5px 10px rgba(0,0,0,0.1);
}

.btn-primary {
    background-color: #0984e3;
    color: white;
}

.btn-success {
    background-color: #00b894;
    color: white;
}
```

Файл index.css:

```
.diary-header {
    display: flex;
    justify-content: space-between;
    align-items: center;
    margin-bottom: 30px;
    padding-bottom: 15px;
    border-bottom: 2px solid #eee;
}

.btn-create {
    background-color: #00b894;
    color: white;
    text-decoration: none;
    padding: 10px 20px;
    border-radius: 8px;
    font-weight: bold;
    transition: 0.3s;
}
```

```
.diary-grid {
    display: grid;
    grid-template-columns: repeat(auto-fill, minmax(300px, 1fr));
    gap: 20px;
}

.diary-card {
    background: #fff;
    border-radius: 12px;
    padding: 20px;
    box-shadow: 0 4px 12px rgba(0,0,0,0.05);
    display: flex;
    flex-direction: column;
    justify-content: space-between;
    transition: transform 0.2s;
}

.diary-card:hover {
    transform: translateY(-5px);
}

.card-content h3 {
    margin-top: 0;
    color: #2d3436;
}

.card-content p {
    color: #636e72;
    line-height: 1.5;
    font-size: 0.95rem;
}

.card-content small {
    color: #b2bec3;
    display: block;
    margin-top: 10px;
}

.card-actions {
    margin-top: 20px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    border-top: 1px solid #f1f2f6;
    padding-top: 15px;
}

.btn-edit {
```

```
    color: #0984e3;
    text-decoration: none;
    font-weight: bold;
}

.btn-delete {
    background: none;
    border: none;
    color: #ff7675;
    cursor: pointer;
    font-weight: bold;
    padding: 0;
}

.btn-delete:hover {
    text-decoration: underline;
}

.empty-state {
    text-align: center;
    padding: 50px;
    color: #888;
    background: #fff;
    border-radius: 12px;
}
```

Файл login.css:

```
.auth-container {
    display: flex;
    justify-content: center;
    align-items: center;
    min-height: 70vh;
}

.auth-card {
    background: #fff;
    padding: 40px;
    border-radius: 15px;
    box-shadow: 0 10px 30px rgba(0,0,0,0.1);
    width: 100%;
    max-width: 400px;
}

.auth-title {
    text-align: center;
    margin-bottom: 25px;
```

```
    color: #2d3436;
    font-size: 1.8rem;
}

.auth-error {
    background-color: #ffeaa7;
    color: #d63031;
    padding: 10px;
    border-radius: 8px;
    margin-bottom: 20px;
    text-align: center;
    font-size: 0.9rem;
    border: 1px solid #fab1a0;
}

.form-group {
    margin-bottom: 20px;
}

.form-group label {
    display: block;
    margin-bottom: 8px;
    font-weight: 600;
    color: #444;
}

.auth-form input {
    width: 100%;
    padding: 12px 15px;
    border: 1px solid #ddd;
    border-radius: 8px;
    font-size: 16px;
    box-sizing: border-box;
    transition: border-color 0.3s;
}

.auth-form input:focus {
    border-color: #0984e3;
    outline: none;
}

.btn-login {
    width: 100%;
    padding: 14px;
    background-color: #0984e3;
    color: white;
    border: none;
    border-radius: 8px;
    font-size: 16px;
```

```
    font-weight: bold;
    cursor: pointer;
    transition: background 0.3s, transform 0.2s;
}

.btn-login:hover {
    background-color: #0a73c5;
    transform: translateY(-1px);
}

.auth-footer {
    margin-top: 25px;
    text-align: center;
    font-size: 0.9rem;
    color: #636e72;
}

.auth-footer a {
    color: #0984e3;
    text-decoration: none;
    font-weight: bold;
}

.auth-footer a:hover {
    text-decoration: underline;
}
```

Файл register.css:

```
.auth-container {
    display: flex;
    justify-content: center;
    align-items: center;
    min-height: 75vh;
    padding: 20px;
}

.auth-card {
    background: #fff;
    padding: 40px;
    border-radius: 15px;
    box-shadow: 0 10px 30px rgba(0,0,0,0.1);
    width: 100%;
    max-width: 450px;
}

.auth-title {
```

```
    text-align: center;
    margin-bottom: 25px;
    color: #2d3436;
    font-size: 1.8rem;
}

.form-group {
    margin-bottom: 18px;
}

.form-group label {
    display: block;
    margin-bottom: 8px;
    font-weight: 600;
    color: #444;
}

.auth-form input {
    width: 100%;
    padding: 12px;
    border: 1px solid #ddd;
    border-radius: 8px;
    box-sizing: border-box;
    font-size: 16px;
}

.auth-form input:focus {
    border-color: #00b894;
    outline: none;
    box-shadow: 0 0 5px rgba(0, 184, 148, 0.2);
}

.btn-register {
    width: 100%;
    padding: 14px;
    background-color: #00b894;
    color: white;
    border: none;
    border-radius: 8px;
    font-size: 16px;
    font-weight: bold;
    cursor: pointer;
    transition: background 0.3s;
    margin-top: 10px;
}

.btn-register:hover {
    background-color: #00947a;
}
```

```
.auth-error {
    background-color: #ffeaa7;
    color: #d63031;
    padding: 10px;
    border-radius: 8px;
    margin-bottom: 20px;
    text-align: center;
}

.auth-footer {
    margin-top: 20px;
    text-align: center;
    font-size: 0.9rem;
}

.auth-footer a {
    color: #00b894;
    text-decoration: none;
    font-weight: bold;
}
```

Файл style.css:

```
:root {
    --primary: #4a90e2;
    --bg: #f4f7f6;
    --text: #333;
    --white: #ffffff;
    --danger: #e74c3c;
    --success: #2ecc71;
}

body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background-color: var(--bg);
    color: var(--text);
    line-height: 1.6;
    margin: 0;
    padding: 0;
}

.container {
    max-width: 800px;
    margin: 20px auto;
    padding: 20px;
    background: var(--white);
```

```
    box-shadow: 0 4px 6px rgba(0,0,0,0.1);
    border-radius: 8px;
}

header {
    background: #333;
    color: white;
    padding: 1rem;
    display: flex;
    justify-content: space-between;
    align-items: center;
}

header a { color: #fff; text-decoration: none; margin-left: 15px; }

.btn {
    display: inline-block;
    padding: 10px 20px;
    border-radius: 5px;
    text-decoration: none;
    cursor: pointer;
    border: none;
    transition: 0.3s;
}

.btn-primary { background: var(--primary); color: white; }
.btn-danger { background: var(--danger); color: white; }
.btn-success { background: var(--success); color: white; }

.entry-card {
    border-bottom: 1px solid #eee;
    padding: 20px 0;
}

.entry-card:last-child { border: none; }

input, textarea {
    width: 100%;
    padding: 12px;
    margin: 10px 0;
    border: 1px solid #ddd;
    border-radius: 4px;
    box-sizing: border-box;
}

.auth-card {
    max-width: 400px;
    margin: 50px auto;
    padding: 30px;
```

```
    background: var(--white);
    border-radius: 12px;
    box-shadow: 0 10px 25px rgba(0,0,0,0.1);
}

.auth-card h2 {
    text-align: center;
    margin-bottom: 25px;
    color: var(--primary);
}

.auth-card .btn {
    width: 100%;
    font-size: 16px;
    margin-top: 10px;
}

.auth-footer {
    text-align: center;
    margin-top: 20px;
    font-size: 14px;
}
```

Файл .htaccess:

RewriteEngine On

RewriteCond %{REQUEST_FILENAME} !-f

RewriteCond %{REQUEST_FILENAME} !-d

RewriteRule ^(.*)$ index.php?url=$1 [L,QSA]

Файл index.php:

```
<?php
session_start();

require_once __DIR__ . '/../config/config.php';

spl_autoload_register(function ($class) {
    $path = __DIR__ . '/../' . str_replace('\\', DIRECTORY_SEPARATOR, $class) .
'.php';
    if (file_exists($path)) {
        require_once $path;
```

```
    }
});

$router = new Core\Router();
```